

Object Oriented Programming

This document contains critical information about Object Oriented Programming in Python that will be used throughout this course.

Note: The terminology included in this document and a basic understanding of Object Oriented Programming which is course content therefore it may be included on exams or quizzes.

Table of Contents

Introduction	2
Terminology	2
Classes	3
Instance Attributes	3
Class Attributes	4
Dunder Methods	4
__init__	4
__eq__	5
__lt__	6
__str__	6
__repr__	7
Other dunder methods	7
Instance Methods	8
Inheritance	8
Super	9
Method Overriding	9
Collections of objects	10
Shallow Copy vs Deep Copy	12

Introduction

Object-Oriented Programming (OOP) is the term used to describe a programming approach based on **objects** and **classes**. OOP allows us to organize software as a collection of objects that consist of both data and behavior. This is in contrast to conventional procedural programming, which only loosely connects data and behavior.

The Python language allows both functional programming and object-oriented programming styles. It is generally thought that object-oriented programming is the best approach for developing software because it allows for well-designed modular code that is easily maintained and also reusable code that is flexible for new applications.

Terminology

Class - A blueprint for creating objects providing values for state (attributes), and implementations of behavior (methods).

Object - A data structure created using a class as its blueprint.

Instance - An object created from a particular class.

Attribute - A characteristic of an object.

Method - An implementation of the behavior of an object of a particular class.

Inheritance - A mechanism where you can derive a class from another class creating a hierarchy of classes that share a set of attributes and methods.

Child - A class that is derived from another class inheriting all the attributes and methods of the other class. It is sometimes called a subclass.

Parent - A class that has one or more other classes inheriting its attributes and methods. It is sometimes called a superclass.

Alias - A second name for the same piece of data.

Shallow copy - A copy of an object that is created by constructing a new compound object and then storing the references into it to the objects found in the original.

Deep copy - A copy of an object that is created by constructing a new compound object and then, recursively, storing copies into it of the objects found in the original.

Classes

Classes are used to create new user-defined data structures that contain information about a specific thing. In the case of an animal, we could create an `Animal()` class to track properties about an Animal, like its name and its age.

It's important to note that a class just provides structure and organization—it's a blueprint for how something should be defined—but it doesn't actually provide any real content itself. The `Animal()` class may specify that the name and age are necessary for defining an animal, but it will not actually state what a specific animal's name or age is.

While the class is the blueprint, an instance is a copy of the class with actual values, literally an object belonging to a specific class. It's not an idea anymore; it's an actual animal, like a dog named Spot who's eight years old.

Defining a class:

```
>>> class Dog:
...     pass
```

You start with the `class` keyword to indicate that you are creating a class, then you add the name of the class.

Instance Attributes

All classes create objects, and all objects contain characteristics called attributes. An instance attribute is specific to each object, meaning two different dogs made with the same class can have different names and ages. **Instance attributes are not shared.** Every object has its own copy of the instance attribute. Since all Dogs have names and ages, we will define instance attributes to assign the name and age to a Dog when it is initialized.

```
>>> class Dog:
...     def __init__(self, name, age):
...         self.name = name
...         self.age = age
```

Class Attributes

Class attributes belong to the class itself; they will be shared by all the instances. Such attributes are defined in the class body section which conventionally at the top, for readability.

```
>>> class Dog:
...     numLegs = 4
...     numEyes = 2
...     def __init__(self, name, age):
...         self.name = name
...         self.age = age
```

In this example, the class attributes are `numLegs` and `numEyes` which represent the shared characteristics of all dogs. In other words, any dog that is created using the `Dog` class will have 4 legs and 2 eyes, regardless of its name or age. In the examples below, we will exclude class attributes from the snippets provided.

Dunder Methods

Dunder methods (short for double underscore) in Python are the methods having two underscores proceeding and succeeding the method name. These are used to define or call special predefined methods native to the Python language. One common use of these is to provide functionality for an operator such as `>`, `<`, or `==`. Other examples of dunder methods are: `__init__`, `__add__`, `__len__`, `__repr__`, etc.

`__init__`

The dunder `init` method is similar to a constructor in C++, Java, and other object-oriented languages. Constructors are used to initialize the object's state. The task of constructors is to initialize (assign values) to the data members of the class when an object of the class is created. It is run as soon as an object of a class is instantiated thus assigning relevant state characteristics.

The `__init__` is a special method used to initialize (e.g., specify) an object's initial attributes by giving them their default value (or state). This method must always include the `self`-variable, which refers to the object itself (e.g., `Dog`) so that it is able to access the instance attributes for that object.

```
>>> class Dog:
...     def __init__(self, name, age):
...         self.name = name
...         self.age = age
```

self is passed into the dunder method to represent the current instance of the object, particularly for the creation of instance variables. To create an instance of the Dog class and access the attributes defined in the dunder init method we use the following:

```
>>> Max = Dog("max", 5)
>>> Max.age
5
>>> Max.name = "Max!"
>>> Max.name
Max!
```

__eq__

To determine what makes objects equal to each other, we can define the dunder eq method. The dunder eq method is called when the == and != operators are used to compare objects. If a class does not have the dunder eq method defined then the objects are considered equal if and only if the objects refer to the same memory location (i.e. they are aliases).

```
>>> class Dog:
...     def __init__(self, name, age):
...         self.name = name
...         self.age = age
>>> max1 = Dog("Max", 5)
>>> max2 = Dog("Max", 5)
>>> max3 = max1
>>> max1 == max2
False
>>> max1 == max3
True
>>> class Dog:
...     def __init__(self, name, age):
...         self.name = name
...         self.age = age
...
...     def __eq__(self, other):
...         return self.name == other.name and self.age == other.age
>>> max1 = Dog("Max", 5)
>>> max2 = Dog("Max", 5)
>>> max1 == max2
True
>>> max3 = Dog("Max", 2)
>>> max1 == max3
False
```

`__lt__`

The dunder `lt` method is called when the `<` operator is used comparing objects. In our example, we consider one `Dog` object to be less than another if the age of the dog object on the left of the `<` operator is less than the `Dog` object on the right of the `<` operator. The dunder `lt` method is used to define how a list of objects is sorted. This is because the `sorted` function or `sort` method calls the `lt` method on the objects within a sequence. Below we will illustrate how to define the dunder `lt` method and compare objects.

```
>>> class Dog:
...     def __init__(self, name, age):
...         self.name = name
...         self.age = age
...
...     def __lt__(self, other):
...         return self.age < other.age
>>> max = Dog("Max", 5)
>>> biscuit = Dog("Biscuit", 4)
>>> max < biscuit
False
>>> biscuit < max
True
>>> bella = Dog("Bella", 1)
>>> dogs = [max, biscuit, bella]
>>> dogs.sort()
>>> for dog in dogs:
...     print(dog.name)
Bella
Biscuit
Max
```

`__str__`

The `str` dunder method is called when an object is printed or casted to a string. The dunder `str` method is intended to be used for readable output for an end-user. It returns a string that describes the object using the most important of that object's attributes.

```
>>> class Dog:
...     def __str__(self):
...         return f"{self.name} is {self.age} years old."
>>> Max = Dog("Max", 5)
>>> print(Max)
Max is 5 years old.
```

```
>>> str(Max)
"Max is 5 years old."
```

`__repr__`

The `repr` dunder method is called when an object is printed/casted to a string or called from the shell. It also returns a string that describes the object, but its focus is usually on conciseness because the `__repr__` method is also used if the object is printed within another data structure.

```
>>> class Dog:
...     def __init__(self, name, age):
...         self.name = name
...         self.age = age
...
...     def __repr__(self):
...         return f"{self.name}"
>>> Max = Dog("Max", 5)
>>> Biscuit = Dog("Biscuit", 4)
>>> Bella = Dog("Bella", 1)
>>> Max
"Max"
>>> print(Max)
Max
>>> dogs = [Max, Biscuit, Bella]
>>> print(dogs)
[Max, Biscuit, Bella]
```

Other dunder methods

There are many more dunder methods, for example `__add__`, `__len__`, `__call__`, etc. that can be used to further the functionality of a class, however the implementation is out of scope for this course. For more information visit [this link](#).

Instance Methods

Instance methods are functions that are defined within a class that are called on the object. Instance methods are used to define behaviors or actions for an object. An example of an instance method is the `sort` method of the `list` class. The `sort` method is an action that can be performed on any list object. Similarly, we can expand the `Dog` class defined above to include a `bark` instance method. The example below is trivial, however, instance methods are often used to modify the object's state and call additional methods.

```
>>> class Dog:
...     def __init__(self, name, age):
...         self.name = name
...         self.age = age
...     def bark(self):
...         print(f"THWG!!")
>>> max = Dog("Max", 5)
>>> max.bark()
THWG!!
```

Inheritance

Inheritance allows us to define a class that inherits all the methods and properties from another class. The parent class is the base class that the child inherits from and the child class is the derived class that extends the parent. The motivation behind inheritance is to reuse code. Parent classes are created that contain all the attributes and methods that are needed for the group, while child classes inherit these attributes and methods while also including attributes and methods that are specializations for a specific type of that class.

For example, a parent class might be `Canine`, with child classes `Dog` and `Wolf`. All `Canines` have a weight, age, and birthday and can bark, including `Dog` and `Wolf`. However, while pet `Dogs` have owners, `Wolves` have packs and ranks. Some other types of `Canines` may have neither of these attributes. In this example, we see that `Dogs` can reuse some of the attributes and methods from the `Canine` class using inheritance.

Now, to start building this example we'll define the `Canine` class and `Dog` class which inherits only the *methods* from the `Canine` class.

```
>>> class Canine:
...     def __init__(self, weight, age):
...         self.weight = weight
...         self.age = age
...     def birthday(self):
...         self.age += 1
```

```

...     def bark(self):
...         print("THWG!!")
>>> class Dog(Canine):
...     def __init__(self, name):
...         self.name = name
>>> Max = Dog("Max")
>>> Max.bark()
THWG!!
>>> Max.birthday()
AttributeError: 'Dog' object has no attribute 'age'

```

Notice that the birthday method throws an attribute error when called. This is because the Dog object only inherited the methods from the Canine class, not the attributes. To inherit the attributes from the parent we will use the super keyword, which we'll discuss in the next section.

Super

super() is used to call the methods from parent class in the child class. super() can be used to call the parents dunder init method and therefore initialize attributes for an object. To avoid the error mentioned previously, we adjust the Dog class to the following:

```

>>> class Dog(Canine):
...     def __init__(self, weight, age, name):
...         super().__init__(weight, age)
...         self.name = name
>>> Max = Dog(60, 5, "Max")
>>> Max.bark()
THWG!!
>>> Max.birthday()
>>> Max.age
6

```

Method Overriding

Sometimes, it is desirable to override or assign different functionality to a method inherited from the parent class. Method overriding occurs when a child class provides more specialization by defining a method with the same name as the parent, thus fitting the child class more closely. Back to our Canine and Dog example, maybe a Dog eats a delicious bone for its birthday which causes its weight to increase by 10%. In the example below, we will override the birthday method in the Dog class so that the age still increases by 1 but the weight also increases by 10%.

```

>>> class Dog(Canine):

```

```

...     def __init__(self, weight, age, name):
...         super().__init__(weight, age)
...         self.name = name
...     def birthday(self):
...         super().birthday()
...         self.weight *= 1.10
>>> Max = Dog(60, 5, "Max")
>>> Max.birthday()
>>> Max.age
6
>>> Max.weight
66.0

```

Notice that although the child class has the same method defined as the parent, the parent method can be accessed using `super()`.

Collections of objects

When building applications and using OOP, it is sometimes useful to use nested objects. Nested objects or collections of objects occur when one or more objects are stored within another object. To provide an example of this, let's build on our previous example so that we have a collection of Dogs or more formally a `Kennel`. We'll design our `Kennel` so that it has a fixed capacity, meaning when a dog is checked in we need to verify that there is room. Additionally, we'll assume a fixed cost of 100 dollars for keeping a dog, but we'll only charge this amount if the dog has been checked in.

```

>>> class Dog(Canine):
...     def __init__(self, weight, age, name):
...         super().__init__(weight, age)
...         self.name = name
...     def birthday(self):
...         super().birthday()
...         self.weight = 1.10 * self.weight
>>> class Kennel():
...     def __init__(self, capacity):
...         self.capacity = capacity
...         self.dogs = []
...         self.revenue = 0
...     def check_in(self, dog):
...         if len(self.dogs) < self.capacity:
...             self.dogs.append(dog)
...         else:
...             print("Sorry we're full!")

```

```

...     def check_out(self, dog):
...         if dog in self.dogs:
...             self.dogs.remove(dog)
...             self.revenue += 100
...         else:
...             print("Cannot check out a dog that isn't here!")

```

Now that we've defined the Kennel class, we can create Dog objects to add to the Kennel and test the functionality.

```

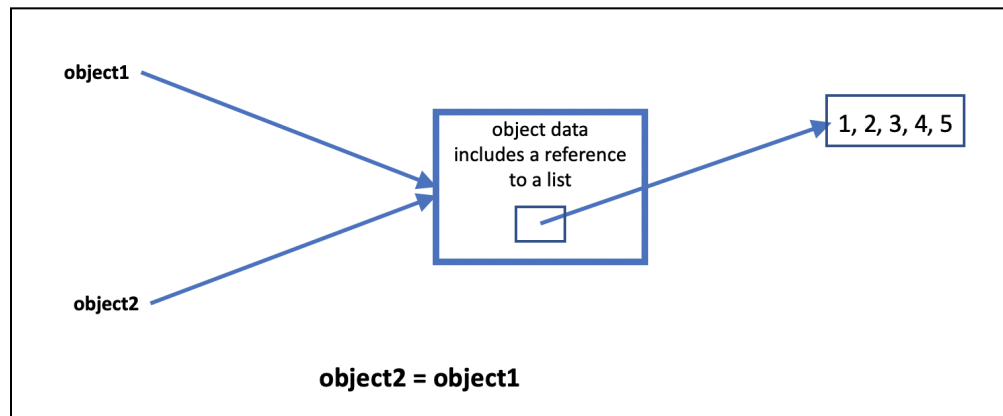
>>> Max = Dog(60, 5, "Max")
>>> Biscuit = Dog(25, 4, "Biscuit")
>>> Bella = Dog(13, 1, "Bella")
>>> Sam = Dog(40, 9, "Sam")
>>> myKennel = Kennel(3)
>>> myKennel.check_in(Max)
>>> myKennel.check_in(Biscuit)
>>> myKennel.check_in(Bella)
>>> myKennel.check_in(Sam)
Sorry we're full!
>>> myKennel.check_out(Sam)
Cannot check out a dog that isn't here!
>>> myKennel.check_out(Max)
>>> myKennel.revenue
100

```

The same concept scales to create structure and organization to model real-world systems. For example, we could have a collection of Dogs in a Kennel and a collection of Kennel's in a Region, and a collection of Regions for a Company. Python could be used to store this data in objects that communicate with a database and send the same content to a web page so users can find a Kennel for their dog!

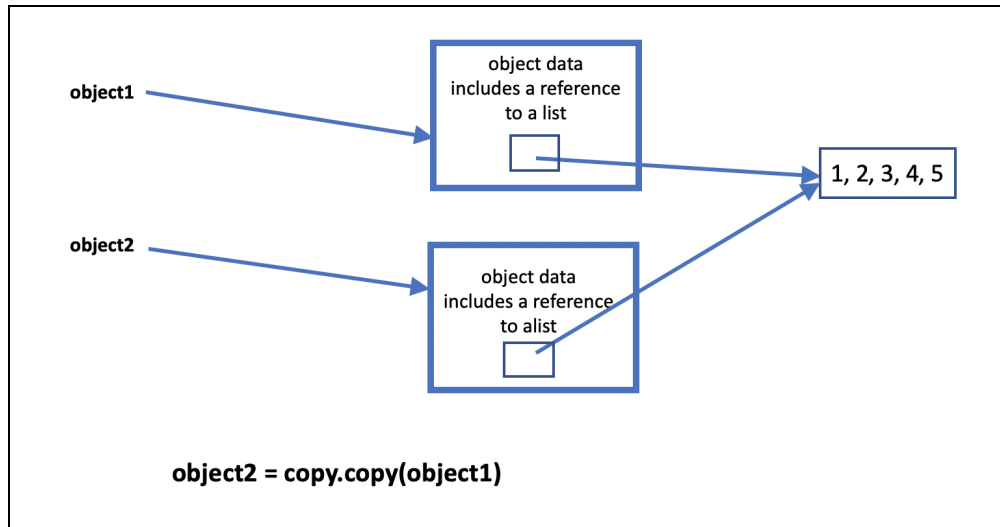
Shallow Copy vs Deep Copy

How do we handle times that we need a copy of an object? The answer depends upon the reason for wanting a copy. It is important to remember that objects are stored as references to memory locations. Assigning a variable to another variable simply is assigning that same reference to another object--doing this assignment is called creating an **alias**. Sometimes an alias, or renaming of the original object, is all you need.

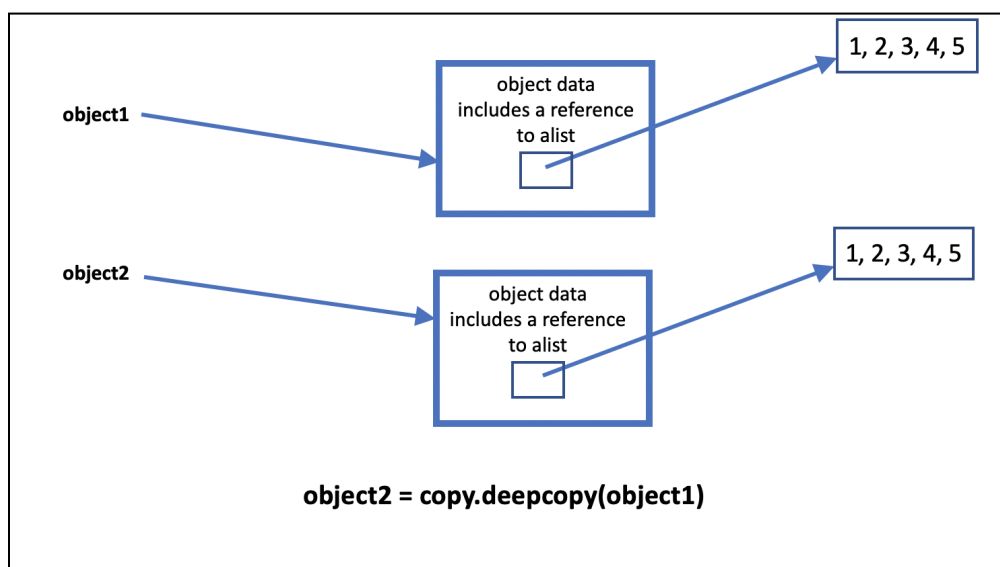


Passing an object as an argument to a function creates an alias as well-- the internal name of the parameter as it is called within the function. During parameter passing, a copy of the object is not actually being made. What is happening is that the reference to the object is passed to the function. Because of this, anything that happens during the function has the potential to change the value of the argument itself.

Importing the copy module provides a way to create a true copy of an object rather than simply an alias. This module provides two copy methods: **copy**, which copies the first level of the object, and **deepcopy**, which recursively copies the object and its contents as many levels deep as are needed. The pictures below illustrate the difference between these two methods.



The image above shows an example of shallow copy, created by using `copy.copy`. A shallow copy is a copy of an object that is created by constructing a new compound object then storing the references into it to the objects found in the original.



The image above shows an example of deepcopy. A deepcopy is a copy of an object that is created by constructing a new compound object and then, recursively, storing copies into it of the objects themselves (not simply the references) found in the original.